DOCUMENTATION COMPLÈTE

Professional Profile Explorer

Plateforme Streamlit — cinq modèles intégrés

Table des matières

1. Présentation
2. Principes d'intégration
3. Architecture générale
4. Organisation du dossier
5. Installation sur une autre machine
6. Bibliothèques
7. Chargement des modèles
8. Profil saisi
9. Chaîne de prédiction
10. Module 1 — Ahmed Bejaoui
11. Module 2 — Ahmed Guesmi
12. Module 3 — Nour Trabelsi
13. Module 4 — Arij Yousfi
14. Module 5 — Seifeddine Alaimi
15. Origine des fichiers de modèles
16. Reproduction des modèles
17. Absence d'endpoints personnalisés
18. Portabilité
19. Utilisation sur le réseau local
20. Tests réalisés
21. Dépannage
22. Limites
23. Confidentialité
24. Maintenance
25. Résumé du démarrage

1. Présentation

Professional Profile Explorer est une application Streamlit qui réunit les cinq modules de machine learning du groupe dans une seule carte profil.

L'utilisateur saisit un profil socio-professionnel. La plateforme prépare ces informations selon les règles propres à chaque notebook, exécute les cinq modèles entraînés et affiche leurs résultats dans cinq cartes.

La plateforme repose sur le jeu de données historique **Adult / Census Income de 1994**. Elle constitue un prototype pédagogique d'exploration. Elle ne doit pas être utilisée comme outil de recrutement, de décision administrative ou de garantie sur la situation réelle d'une personne.

2. Principes d'intégration

L'intégration respecte les règles suivantes :

- aucun notebook du dossier `final notebooks` n'est modifié ;
- les hyperparamètres, variables et règles de feature engineering des modèles restent ceux des notebooks ;
- les modèles sont chargés depuis des fichiers portables `.joblib` ;
- les cinq prédictions sont exécutées dans le même processus Python ;
- aucun endpoint API personnalisé n'est nécessaire ;
- le revenu estimé par le module 1 est transmis automatiquement aux modules qui utilisent la variable `income` ;
- les modèles sont chargés une seule fois grâce au cache Streamlit.

3. Architecture générale

```
Navigateur
    |
    | interface Streamlit
    v
streamlit_app.py
    |
    +-- collecte les champs du profil
    +-- crée une ligne pandas DataFrame
    +-- exécute Ahmed Bejaoui en premier
    +-- transmet le revenu estimé aux autres modules
    |
    +--> ahmed_bejaoui.joblib
    +--> ahmed_guesmi.joblib
    +--> nour_trabelsi.joblib
    +--> arij_yousfi.joblib
    +--> seifeddine_alaimi.joblib
    |
    v
Cinq cartes de résultats
```

Tout se déroule localement dans le processus Streamlit. Les fonctions Python appellent directement `predict()` ou `predict_proba()` sur les modèles.

4. Organisation du dossier

```
platform/
├── .streamlit/
│   └── config.toml
├── models/
│   ├── ahmed_bejaoui.joblib
│   ├── ahmed_guesmi.joblib
│   ├── nour_trabelsi.joblib
│   ├── arij_yousfi.joblib
│   └── seifeddine_alaimi.joblib
├── DOCUMENTATION.md
├── README.md
├── requirements.txt
├── run_platform.py
├── streamlit_app.py
└── build_models.py
```

Les anciens fichiers `app.py`, `engine.py`, `templates/` et `static/` appartiennent à la précédente version Flask. Ils ne sont pas appelés par la version Streamlit.

Rôle des fichiers actifs

`streamlit_app.py`

Point d'entrée fonctionnel de la plateforme :

- définit les valeurs proposées dans les listes ;
- charge les cinq modèles ;
- affiche le formulaire ;
- prépare les variables propres à chaque modèle ;
- exécute les prédictions ;
- affiche les résultats.

`run_platform.py`

Lance Streamlit avec le même interpréteur Python que celui utilisé pour exécuter la commande :

```
python run_platform.py
```

Cette méthode évite de dépendre d'une commande `streamlit` globale.

`models/`

Contient les cinq modèles sérialisés nécessaires à l'inférence. Ce dossier doit toujours rester à côté de `streamlit_app.py`.

`requirements.txt`

Fixe les versions des bibliothèques utilisées lors de la création et du chargement des modèles.

`.streamlit/config.toml`

Définit le thème visuel de l'application et le fonctionnement du serveur Streamlit.

`build_models.py`

Script de reproduction des fichiers `.joblib`. Il n'est pas exécuté pendant l'utilisation normale de la plateforme.

5. Installation sur une autre machine

5.1 Prérequis

- Python 3.11 ou plus récent ;
- une connexion Internet uniquement pour installer les bibliothèques ;
- le dossier `platform` complet, y compris son sous-dossier `models`.

Les notebooks et `adult.data` ne sont pas nécessaires pour lancer l'application déjà préparée.

5.2 Installation sur macOS ou Linux

Ouvrir un terminal dans le dossier du projet :

```
cd platform
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python run_platform.py
```

5.3 Installation sur Windows PowerShell

```
cd platform
py -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python run_platform.py
```

Si PowerShell bloque l'activation de l'environnement :

```
Set-ExecutionPolicy -Scope Process Bypass
.\.venv\Scripts\Activate.ps1
```

5.4 Ouverture

Après le démarrage, ouvrir :

```
http://localhost:8501
```

Pour arrêter l'application, revenir au terminal et utiliser `Ctrl+C`.

6. Bibliothèques

La plateforme utilise :

- `streamlit` pour l'interface ;
- `pandas` pour construire les profils et leurs variables ;
- `numpy` pour les calculs numériques ;
- `scikit-learn` pour les pipelines, transformations et K-Means ;
- `xgboost` pour les modèles supervisés ;
- `joblib` pour charger les modèles ;
- `imbalanced-learn` uniquement si les modèles d'Ahmed Guesmi sont reproduits avec `build_models.py`.

Les versions exactes se trouvent dans `requirements.txt`. Il est recommandé de les conserver, car les objets `joblib` contenant des modèles scikit-learn/XGBoost ne sont pas garantis compatibles entre toutes les versions.

7. Chargement des modèles

La fonction `load_models()` associe chaque module à son fichier :

- `income` → `ahmed_bejaoui.joblib` ;
- `occupation` → `ahmed_guesmi.joblib` ;
- `rhythm` → `nour_trabelsi.joblib` ;
- `education` → `arij_yousfi.joblib` ;
- `cluster` → `seifeddine_alaimi.joblib`.

Le décorateur `@st.cache_resource` garde les objets chargés en mémoire. Un changement de champ dans l'interface relance le script Streamlit, mais ne relit pas les modèles sur le disque à chaque fois.

Si un fichier est absent, la carte concernée affiche :

```
Modèle absent du dossier platform/models.
```

8. Profil saisi

La partie gauche de l'interface recueille :

- l'âge, de 17 à 90 ans ;
- le niveau d'études Adult, de `Preschool` à `Doctorate` ;
- les heures de travail hebdomadaires, de 1 à 99 ;

- le secteur d'emploi (`workclass`) ;
- le métier déclaré (`occupation`) ;
- le lien familial (`relationship`) ;
- la situation familiale (`marital-status`) ;
- le gain en capital ;
- la perte en capital ;
- le pays ;
- le sexe et l'origine, dans une section séparée.

Le niveau d'études affiché est converti en `education-num` de 1 à 16.

Le sexe et l'origine sont transmis uniquement au module d'Arij, conformément à son notebook. Les quatre autres modules ne les utilisent pas.

9. Chaîne de prédiction

Étape 1 — Création du profil commun

Les champs de l'interface sont rassemblés dans un DataFrame pandas contenant une seule ligne.

Étape 2 — Revenu estimé

Le modèle d'Ahmed Bejaoui est exécuté en premier. Il produit :

- la probabilité de la classe `>50K` ;
- la classe `<=50K` ou `>50K`, selon son seuil de décision ;
- un libellé lisible dans l'interface.

Étape 3 — Propagation de `income`

La classe de revenu prédite est ajoutée au profil sous la forme :

```
<=50K
```

ou :

```
>50K
```

Cette valeur est ensuite utilisée par les modules d'Ahmed Guesmi, Nour, Arij et Seifeddine lorsque leur modèle attend `income`.

Étape 4 — Exécution des quatre autres modules

Chaque adaptateur applique le feature engineering attendu par son modèle, respecte l'ordre des colonnes d'entraînement, puis exécute la prédiction.

Étape 5 — Affichage

`render_card()` affiche :

- le nom du membre ;
- le titre du module ;
- la classe ou le persona retenu ;
- les probabilités disponibles ;
- une explication courte.

10. Module 1 — Ahmed Bejaoui

Objectif

Classification binaire du revenu :

- `<=50K` ;
- `>50K`.

Modèle

Pipeline XGBoost sauvegardé dans :

```
models/ahmed_bejaoui.joblib
```

Variables utilisées

Variables originales :

- `relationship` ;
- `education-num` ;
- `capital-gain` ;
- `age` ;
- `capital-loss`.

Variables créées :

- `workload` ;

- `workclass-occupation` ;
- `occupation-group` .

Feature engineering

`workload` contient :

- `part-time` pour moins de 35 heures ;
- `full-time` pour 35 à 40 heures ;
- `overtime` au-dessus de 40 heures.

`workclass-occupation` concatène le secteur et le métier.

`occupation-group` regroupe les métiers selon le dictionnaire sauvegardé avec le modèle.

Prédiction

La plateforme appelle :

```
model["pipeline"].predict_proba(...)
```

La barre représente toujours la probabilité de `>50K` .

Le seuil de décision sauvegardé est **0,375** :

- probabilité inférieure à 0,375 → `<=50K` ;
- probabilité supérieure ou égale à 0,375 → `>50K` .

Le résultat de ce module alimente les quatre modules suivants.

11. Module 2 — Ahmed Guesmi

Objectif

Classification multiclasse d'une famille de métiers :

- Bureau, vente & protection ;
- Cadres & professions intellectuelles ;
- Manuel & technique ;
- Services & manutention.

Le module fournit aussi un métier indicatif appartenant à la famille prédite.

Modèles

Le fichier `ahmed_guesmi.joblib` contient :

- un XGBoost pour la famille de métiers ;
- un XGBoost pour les 14 métiers ;
- le dictionnaire métier → famille ;
- les encodeurs de classes ;
- le codage de `workclass` ;
- les seuils de confiance.

Variables utilisées

- `education-num` ;
- `age` ;
- `hours-per-week` ;
- revenu estimé ;
- `workclass` ;
- `education-level` créé ;
- `age-group` créé ;
- `workload` créé.

Le métier saisi n'est pas une entrée du modèle : il correspond à la cible historique que le module cherche à estimer.

Gestion de l'incertitude

- confiance inférieure à 0,53 → `Profil trop incertain` ;
- confiance de 0,53 à moins de 0,68 → fiabilité modérée ;
- confiance supérieure ou égale à 0,68 → fiabilité élevée.

Le métier indicatif est choisi parmi les métiers appartenant à la famille prédite.

12. Module 3 — Nour Trabelsi

Objectif

Classification multiclasse du rythme hebdomadaire probable :

- `Partiel (<35 h)` ;
- `Standard (35-40 h)` ;
- `Plus de 40 h`.

Modèle

XGBoost avec préprocesseur sauvegardé dans :

```
models/nour_trabelsi.joblib
```

Variables utilisées

- `age` ;
- `education-num` ;
- `occupation` ;
- `workclass` ;
- revenu estimé ;
- `relationship` ;
- `age-group` créé ;
- `education-level` créé ;
- `employment-sector` créé.

Point important

`hours-per-week` définit la cible d'entraînement et n'est donc pas utilisé comme entrée de ce modèle. La carte de Nour n'utilise pas la valeur d'heures saisie par l'utilisateur.

Le préprocesseur :

- remplace les valeurs numériques manquantes par la médiane ;
- standardise les variables numériques ;
- remplace les catégories manquantes par la modalité la plus fréquente ;
- applique un One-Hot Encoding ;
- conserve l'ordre exact des variables appris par XGBoost.

La plateforme affiche les trois probabilités dans l'ordre décroissant.

13. Module 4 — Arij Yousfi

Objectif

Classification binaire du niveau d'études historiquement associé au profil :

- `Sous la licence` ;
- `Licence ou plus`.

La cible `Licence ou plus` correspond à `education-num >= 13`.

Modèle

XGBoost sauvegardé dans :

```
models/arij_yousfi.joblib
```

Préparation

La plateforme reproduit les transformations du notebook :

- revenu en 0/1 ;
- sexe en 0/1 ;
- tranche d'âge ;
- tranche de charge de travail ;
- secteur d'emploi ;
- présence d'un capital ;
- One-Hot Encoding des variables nominales ;
- réalignement sur les 39 colonnes retenues pendant l'entraînement.

Les colonnes absentes du profil deviennent 0 grâce à :

```
reindex(columns=model["columns"], fill_value=0)
```

Point important

Le niveau d'études sélectionné dans le formulaire n'est pas transmis à ce module, car il représente la cible à prédire. Il reste utilisé par les autres modules.

Le seuil de décision est 0,50.

Le sexe et l'origine sont utilisés par ce module parce qu'ils figurent dans le modèle final de son notebook. L'interface les place dans une section séparée pour rendre cette utilisation visible.

14. Module 5 — Seifeddine Alaimi

Objectif

Segmentation non supervisée des profils professionnels avec K-Means.

Modèle

Le fichier `seifeddine_alaimi.joblib` contient :

- le préprocesseur ;
- le K-Means ;
- la correspondance entre labels techniques et groupes affichés ;
- les noms des personas ;
- les bornes des tranches d'âge et d'heures ;
- l'ordre des colonnes.

Variables utilisées

Numériques :

- `age` ;
- `education-num` ;
- `hours-per-week` ;
- `net_capital_log` ;
- `work_life_ratio`.

Ordinales :

- `age_group` ;
- `workload`.

Catégorielles :

- `occupation` ;
- `workclass` ;
- revenu estimé.

Variables créées

```
net_capital = capital-gain - capital-loss
```

`net_capital_log` applique une transformation logarithmique signée.

```
work_life_ratio = hours-per-week / age
```

Personas

Les quatre groupes sont :

- Actifs confirmés à temps plein ;
- Jeunes actifs en début de carrière ;
- Professionnels suractifs ;
- Hauts revenus à fort capital.

Le sexe et l'origine ne participent pas à la formation des groupes.

15. Origine des fichiers de modèles

Modèles déjà exportés par leurs notebooks

Les modèles suivants existaient déjà sous forme `.joblib` :

- Ahmed Bejaoui ;
- Nour Trabelsi.

Ils ont été copiés dans `platform/models` sous des noms uniformes.

Notebooks sans artefact d'inférence complet

Les notebooks d'Ahmed Guesmi, Arij Yousfi et Seifeddine Alaimi ne laissaient pas tous un fichier unique directement utilisable par la plateforme.

`build_models.py` reprend leurs règles finales, leurs hyperparamètres, leurs variables et leurs graines aléatoires afin de produire les fichiers portables. Cette opération ne modifie pas les notebooks.

16. Reproduction des modèles

Cette section n'est utile que si un fichier du dossier `models` est perdu ou si le groupe décide volontairement de régénérer les artefacts.

Prérequis supplémentaires à la racine du projet :

- `adult/adult.data` ;
- `member1_xgboost_best.joblib` ;
- `member3_rhythm_xgboost.joblib`.

Commande :

```
cd platform
python build_models.py
```

Cette commande remplace les cinq fichiers dans `platform/models`. Elle ne doit pas être exécutée pour simplement lancer l'application.

17. Absence d'endpoints personnalisés

La version Streamlit ne définit pas de routes telles que :

```
/api/profile
/api/status
/api/choices
```

Ces routes appartenaient à l'ancienne version Flask.

Streamlit gère lui-même la communication HTTP/WebSocket avec le navigateur. Le code du projet ne définit aucune API REST : les modèles sont appelés directement par des fonctions Python dans `streamlit_app.py`.

18. Portabilité

Pour transférer la plateforme à un membre du groupe :

1. compresser ou copier le dossier `platform` complet ;
2. vérifier que les cinq fichiers existent dans `platform/models` ;
3. installer les dépendances depuis `requirements.txt` ;
4. exécuter `python run_platform.py`.

Il ne faut pas copier seulement `streamlit_app.py`, car l'application a besoin du dossier `models`, du fichier de configuration et des dépendances.

19. Utilisation sur le réseau local

Pour rendre temporairement l'application accessible aux personnes connectées au même réseau Wi-Fi :

```
python -m streamlit run streamlit_app.py --server.address 0.0.0.0
```

Streamlit affiche alors une `Network URL`. Les autres membres peuvent ouvrir cette adresse si le pare-feu de la machine l'autorise.

Cette méthode ne doit pas être utilisée pour exposer directement l'application sur Internet sans authentification, HTTPS et configuration serveur adaptée.

20. Tests réalisés

L'intégration a été testée de deux façons :

- appel direct des cinq fonctions de prédiction ;
- vérification visuelle dans un navigateur sur `localhost:8501`.

Profil courant

- 35 ans ;
- HS-grad ;
- 40 heures ;
- Private ;
- Craft-repair ;
- capital nul.

Résultats observés :

- revenu : 50 000 $ ou moins ;
- famille de métiers : profil trop incertain ;
- rythme : Standard ;
- éducation associée : Sous la licence ;
- persona : Jeunes actifs en début de carrière.

Profil à capital extrême

- 22 ans ;
- Bachelors ;
- 40 heures ;
- Sales ;
- capital-gain de 99 999.

Résultats observés :

- revenu : Plus de 50 000 $;
- famille : Cadres & professions intellectuelles, avec fiabilité modérée ;
- rythme : Standard ;
- éducation associée : Sous la licence ;
- persona : Hauts revenus à fort capital.

Profil cadre

- 45 ans ;

- Masters ;
- 50 heures ;
- Exec-managerial.

Résultats observés :

- revenu : Plus de 50 000 $;
- famille : Cadres & professions intellectuelles ;
- rythme : Plus de 40 heures ;
- éducation associée : Licence ou plus ;
- persona : Professionnels suractifs.

Ces scénarios vérifient le chargement des cinq artefacts, le feature engineering et la propagation du revenu.

21. Dépannage

`ModuleNotFoundError`

Cause probable : les dépendances ne sont pas installées dans l'interpréteur Python actif.

Solution :

```
python -m pip install -r requirements.txt
```

Vérifier également que l'environnement virtuel est activé.

`Modèle absent du dossier platform/models`

Vérifier la présence des cinq fichiers `.joblib`. Le dossier `models` doit être au même niveau que `streamlit_app.py`.

Erreur de compatibilité Joblib, scikit-learn ou XGBoost

Créer un nouvel environnement virtuel puis installer exactement les versions de `requirements.txt`.

Le port 8501 est déjà utilisé

Lancer sur un autre port :

```
python -m streamlit run streamlit_app.py --server.port 8502
```

Puis ouvrir :

```
http://localhost:8502
```

La page ne se met pas à jour

Arrêter le serveur avec `Ctrl+C`, le relancer, puis actualiser le navigateur. Le cache peut aussi être vidé depuis le menu Streamlit.

Le navigateur ne s'ouvre pas automatiquement

Ouvrir manuellement :

```
http://localhost:8501
```

22. Limites

- Les données Adult datent de 1994.
- Les résultats décrivent des associations historiques, pas des causalités.
- La première prédiction de revenu influence quatre cartes suivantes ; une erreur du premier module peut donc se propager.
- Une probabilité n'est pas une certitude individuelle.
- Le module d'Ahmed Guesmi peut s'abstenir lorsque la confiance est trop faible.
- Un cluster est un groupe de proximité statistique, pas une vérité sur une personne.
- Les catégories proposées restent limitées aux modalités apprises dans Adult.
- Les objets `.joblib` doivent être considérés comme du code Python : charger uniquement les fichiers fournis par le groupe.
- Le modèle d'Arij utilise des variables sensibles présentes dans son notebook ; cette utilisation doit être discutée dans l'analyse d'équité.

23. Confidentialité

La plateforme ne contient pas de base de données et n'enregistre pas les profils dans un fichier. Les valeurs sont conservées dans la session Streamlit pendant l'utilisation.

Pour un usage autre que la démonstration locale, il faudrait ajouter :

- une politique de confidentialité ;
- une authentification ;
- HTTPS ;
- une gestion explicite des journaux ;

- des contrôles d'accès ;
- un hébergement sécurisé.

24. Maintenance

Remplacer un modèle

Un fichier peut être remplacé seulement si son nouveau contenu respecte le contrat attendu par sa fonction dans `streamlit_app.py`.

Par exemple, le modèle de Nour doit conserver :

- `model` ;
- `preprocessor` ;
- `features` ;
- `feature_names` ;
- `labels`.

Une modification de clés, de noms de colonnes ou d'ordre des variables exige une adaptation du code d'inférence.

Ajouter un module

Pour ajouter un sixième module :

1. exporter son modèle en `.joblib` ;
2. l'ajouter dans `load_models()` ;
3. créer une fonction d'adaptation et de prédiction ;
4. ajouter une carte dans la liste `cards` ;
5. tester au moins un profil courant et un profil extrême.

25. Résumé du démarrage

Sur toute machine disposant de Python :

```
cd platform
python -m pip install -r requirements.txt
python run_platform.py
```

Adresse :

```
http://localhost:8501
```

La plateforme fonctionne ensuite sans réexécuter les notebooks et sans réentraîner les modèles.